

13th Place Solution - XGBoost ensemble

Rank	13
Team	G&G
Author	Gabor Balazs
Topic ID	549781
Version	Original

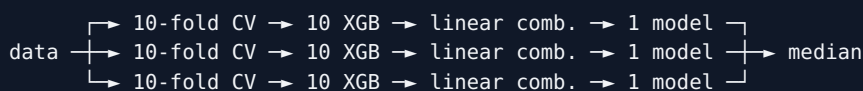
Source: <https://www.kaggle.com/competitions/um-game-playing-strength-of-mcts-variants/discussion/549781>
Retrieved with Kaggle CLI on 2026-07-03.

13th Place Solution for the UM - Game-Playing Strength of MCTS Variants Competition

I would like to thank all organizers and participants for the fantastic experience of this competition. It was unexpected and quite lucky that I ended up at the 13rd place because I managed to pick my only model which scored 0.425 on the PB out of the many with equal LB score of 0.420 (they all scored 0.426 on PB except the winner one).

Overview of the approach

My final model was taking the median of three ensemble models each formed by a linear combination of 10 XGBoost (XGB) regressors coming from a 10-fold CV. In a picture:



I also used agent-flip data augmentation, some feature engineering, and tried to avoid overfitting by bagging, column sub-sampling, and early stopping during boosting. All XGB models were also clipped to the target range $[-1,1]$.

Details of the submission

I dropped all null and zero variance features. I also dropped columns sequentially with correlation to another at least 95%.

Agent-flip data augmentation

Similarly to others, I also swapped the `agent1` and `agent2` columns of the `train` data, inverted the `AdvantageP1` column as `1-train['AdvantageP1']`, and flipped the sign on the target `utility_agent1` (I did not use the `num_wins_agent1` and `num_losses_agent1` fields at the end). At first, and for the training of the final model, I only used this augmentation for the symmetric games (`train['Asymmetric'] == 0`) as I was afraid to create "too unrealistic" scenarios. Later I performed the augmentation for all the data, but my linear combination of XGBs could only reach 0.422 that way (I admit I tuned this significantly less). The second model I submitted to the final (an ensemble of 10-10 LGB and XGB models) was trained on all augmented data and its LB score 0.421 ended up 0.428 on the PB.

Feature engineering

The only feature which differentiated the game play experience of the agents was `AdvantageP1`. At least I read the [ludii code](#) of the other random play behavior metrics and I found them being symmetric to the agents. Because I used quite aggressive column sub-sampling for boosting to prevent game memorization, I wanted to ensure that this feature can be chosen with a higher probability. Hence I added the following variations by mixing it with features of high `gain` importance (`X` was the processed `train` data):

```
X = X.with_columns([
    pl.Series('SgnAdvP1MulGameTreeComplexity',
              (X['AdvantageP1']-0.5) * X['GameTreeComplexity']).clip(-300, 300),
    pl.Series('SgnAdvP1MulBalance', (X['AdvantageP1']-0.5) * X['Balance']),
    pl.Series('SgnAdvP1DivVariance1',
              (X['AdvantageP1']-0.5) / (2.0-X['OutcomeUniformity'].clip(0,1))),
    pl.Series('SgnAdvP1DivDrawFrequency',
              (X['AdvantageP1']-0.5) / (1.0+X['DrawFrequency'].clip(0,1))),
    pl.Series('SgnAdvP1MulCompletion', ((X['AdvantageP1']-0.5) * X['Completion'])),
    pl.Series('SgnAdvP1PerPlayoutsPerSeconds',
              (X['AdvantageP1']-0.5) / (X['PlayoutsPerSecond'] + 1e-4)).clip(-0.5, 0.5),
    pl.Series('SgnAdvP1DivDurationTurnsNotTimeouts',
              (X['AdvantageP1']-0.5) / (X['DurationTurnsNotTimeouts']+1e-6)).clip(-0.5, 0.5),
    pl.Series('SgnAdvP1MulBranchingFactorMedian',
              (X['AdvantageP1']-0.5) * X['BranchingFactorMedian']).clip(-90, 90),
    pl.Series('SgnAdvP1MulBranchingFactorAverage',
              (X['AdvantageP1']-0.5) * X['BranchingFactorAverage']).clip(-100, 100),
])
```

I dropped many features which mostly described the physical characteristics of the board because they managed to show up relatively high (top 50-100) on the `gain` feature importance metrics and I did not want to believe that they can generalize well:

```
X = X.drop([
    'PieceNumberAverage', 'PieceNumberMaximum', 'NumPlayableSitesOnBoard'
    'NumColumns', 'NumRows', 'NumCorners', 'NumOuterSites', 'NumLayers',
    'NumVertices', 'NumTopSites', 'NumRightSites', 'NumCentreSites',
    'NumConvexCorners', 'NumConcaveCorners', 'NumPhasesBoard',
    'NumContainers', 'NumComponentsType', 'NumStartComponentsBoard',
    'NumStartComponentsHand', 'NumStartComponents',
])
```

I also used these features to avoid losing all information by the previous drops:

```
X = X.with_columns([
    pl.Series('IsSquaredBoard', X['NumRows'] == X['NumColumns']),
    pl.Series('PlayoutsPerMoves',
              X['PlayoutsPerSecond'] / (X['MovesPerSecond']+1e-6)).clip(0, 1),
    pl.Series('PieceNumberRatio', X['PieceNumberAverage'] / X['PieceNumberMaximum']),
    pl.Series('AverageTurns', X['NumPlayableSitesOnBoard'] / X['PieceNumberAverage']),
    pl.Series('ActionsPerTurn', X['DurationActions'] / X['DurationTurns']).clip(1, 300),
])
```

Finally, I also dropped the `GameRulesetName`, `EnglishRules`, and `LudRules` columns. Unfortunately I could not make use of them.

Boosting

I used `StratifiedGroupKFold` for 10-fold CV, grouped by `GameRulesetName`, and used the integer class labels `(y_data*10).round(decimals=0).astype(int)` for stratification. The parameters of my the three boosting XGB regressors were quite similar (the difference is shown in the brackets):

```
max_leaves=[250, 200, 250], max_depth=[35, 30, 35],
n_estimators=[3500, 4000, 3500], learning_rate=[0.04, 0.03, 0.04],
subsample=[0.9, 0.75, 0.9], colsample_bylevel=0.5, colsample_bynode=0.25,
min_child_weight=25, min_split_loss=1e-4, reg_lambda=2.0, reg_alpha=4.0,
max_bin=64, max_cat_threshold=32, grow_policy='lossguide', enable_categorical=True,
```

I used many leaves and quite a depth, so I tried to counteract the potential overfitting with column sub-sampling which left only $0.5 * 0.25 = 0.125$ portion of the features for each split. The regularization parameters came from early grid search runs, but I only tuned the parameters in the top three rows at the end. I used 10% of the training data for early stopping with minimum delta $1e-5$ for the first model, and no early stopping for the other two. Every CV training was performed a few times (3-5) with the same data shuffling seed and different estimator seeds, and then I picked the best model for each split to optimize the "CV score". As it helped on the LB too I justified this for myself as a random restart technique. :)

Linear combination (stacking)

I combined the 10 XGB models from the CV runs linearly using weights minimizing the RMSE on all the samples. To take clipping into account, first I calculated all the predictions for all models in `yhats` (so it is an array of 10 rows and as many columns as samples), and then I minimized the RMSE using the following code (`y` contained the training target values):

```
from scipy.optimize import minimize
def fobj(w):
    return (np.mean(np.square(np.clip(np.dot(w, yhats), -1.0, 1.0) - y))
            + 1e5 * min(0, np.min(w))**2)
w = np.ones(yhats.shape[0]) / yhats.shape[0]
res = minimize(fobj, w)
wopt = res.x
rmse_opt = calc_rmse(np.dot(wopt, yhats).clip(-1, 1), y)
```

That is I started the search from the uniform weights, penalized for negative weights, and took clipping into account. The results for the three linear combinations in the final model were:

```
wopt1 = [0.086247, 0.144699, 0.133306, 0.101278, 0.130589,
         0.136385, 0.076667, 0.139516, 0.108264, 0.106763]
wopt2 = [0.1059 , 0.14333 , 0.113467, 0.111721, 0.120991,
         0.137219, 0.097818, 0.12802 , 0.094427, 0.12176 ]
wopt3 = [0.115637, 0.109509, 0.119757, 0.126103, 0.134074,
         0.137645, 0.076969, 0.118834, 0.125846, 0.098289]
```

They sum up to 1.164, 1.175, and 1.163, respectively, so eventually I too pushed higher the prediction values as others mentioned. I did not consider nonlinear combination as I was too afraid of overfitting.

Other things

LightGBM (LGB) and CatBoost (CAT)

I used LGB a lot to tune its parameters, and then I applied similar settings to XGB and CAT. LGB ran more than twice as fast on CPU as the others so I could reserve my GPU quota for the most promising runs. I managed to get LGB and CAT models down to 0.421 on the LB, but I only managed to get 0.420 with XGB models. In fact each XGB combination in the ensemble of my final model reached 0.420, hence I chose this instead of an LGB-XGB-CAT variant (which scored 0.421 each and 0.420 together on the PB, and only 0.426 together on the PB).

Failed attempts

- Similarly to the [9th Place Solution](#), I also tried to augment the data further using transitivity, that is if agent A wins over agent B all the time, and agent B wins over agent C all the time, then agent A should win over agent C all the time. I considered this for symmetric games only, even applying the rule for multiple iterations and dropping all cases for games where contradictions appear, but it did not help the CV and eventually increased the LB score by 0.002.
- I tried to add nearest neighbor features using a few game properties, but I usually ended up with poor CV scores suffering from overfitting.
- As the linear combination weights summed up higher than 1 with a strong and consistent advantage on the LB, I was a bit afraid that the same bias will not be present in the private data. So I was hoping to reduce this by training separate models for symmetric and asymmetric games (much smaller models for the second case), but it always performed much worse on the LB (around 0.428) so I abandoned the idea.
- I tried to clamp the prediction values to the ones appeared in the training data, but it actually increased the CV error for me.
- I tried to use the average of symmetrized predictions (even just for the symmetric games) to do the agent-flip augmentation of test data, flip the sign of the model output, and average this with the normal prediction. But it lowered my LB score by 0.001, so I dropped the idea.
- I also tried to use the `LudRules` column to detect in prediction time whether I have seen the sample in training or not, and use a more aggressive model if so (which could even use the `LudRules` column), but it did not improve my LB score. As the [competition host mentioned](#) there were no overlapping games between the train and test data, so there is no surprise here.

Supplemental Q&A

yunsuxiaozi · 2024-12-04T01:10:02.623000

I saw that you used the median when using the ensemble, may I ask why?

Gabor Balazs · 2024-12-04T08:31:28.203000

Good point, I forgot to mention that I measured the RMSE for the mean and the median combination on the entire sample and the median scored slightly better. There was not a big difference though, the median

scored 0.2005 while the mean was 0.2011. And I did not consider a weighted average due to my fears of overfitting.

Mart Preusse · 2024-12-04T09:00:16.047000

Congratulations! It is amazing that you handled XGB so well, as I could not tame it at all! How did you feed the categorical features to XGB? Did you let XGB handle it on default or did you encode the features? Did you split the agents? Your methods to select and engineer the features makes a lot of sense and also tweaking the hyper parameters with an eye on overfitting. Thanks for sharing your solution.

Gabor Balazs · 2024-12-04T09:54:30.787000

Thank you for the good questions.

I fed pandas data frames to XGBoost with `categorical columns` and passed the parameter `enable_categorical=True`. I think the key XGB parameter to set was `grow_policy='lossguide'` because as I recall the default depth-wise grow policy gave me much worse results.

Indeed I split the agents to ('selection', 'exploration_const', 'payout', 'score_bounds') and encoded them as categorical. In fact I encoded all columns as categorical which had no more distinct values than 8. I also noticed that keeping the original unsplitted agent features helped (with ordinal encoding), so I kept them but only after removing the score_bounds part (which seemed to be the least important based on the feature importance metrics) to make them a bit more robust against overfitting.